HEINRICH-HEINE-UNIVERSITÄT DÜSSELDORF

Bachelor Thesis

Implementierung und Evaluation der Reinforcement-Learning-Methoden Self-Play und Liga-Training in der MicroRTS-Umgebung

Name!!

Degree:	B.Sc. Computer Science
Supervisor:	Professor Dr. Paul Swoboda
Co-Supervisor:	Dr. Peter Arndt
Faculty:	Mathematisch-Naturwissenschaftliche Fakultät

XXXXXXXXXXX

Abstract

This thesis improves an existing agent for the real-time strategy game (RTS) MicroRTS. RTS games are considered a challenging domain for Deep Reinforcement Learning (DRL), since agents must deal with large combinatorial action spaces, simultaneous decisions, as well as delayed and often sparse rewards. As a baseline method, we use Proximal Policy Optimization (PPO). In many DRL training setups, such as pure PPO, one reaches a point at which the agent cannot be trained further efficiently using PPO alone, because there are not enough strong opponents to train against or because the available opponents are not diverse enough to develop a robust policy. An obvious and simple countermeasure is Self-Play (SP) however, in practice it can be unstable and lead to overfitting or cyclical strategies, which can reduce performance against external opponents.

League Training (LT) stabilizes SP by building a population of agents across different training stages and roles. The training agent is then deliberately matched against diverse opponents. Among others, against specialized agents that were trained to exploit weaknesses of the current policy. We evaluate the resulting agents in matches against various opponent types and compare LT with pure SP as well as with our initial agent. Our results show that LT increases the robustness and generalization capability of the agent against different opponents.

Contents

1. Introduction

Reinforcement Learning (RL) has established itself in recent years as a powerful approach for developing agents in complex environments. Game environments such as RTS games pose a particular challenge in this regard, with large action spaces, long-term planning, and simultaneous decisions. However, they also provide clearly defined state spaces, reward structures, and repeatable experiments, which makes them an ideal and challenging test environment for researching RL methods.

MicroRTS MicroRTS is a simplified version of RTS games that was developed specifically for research [Huang et al., 2021]. It is discrete and reduces the visual and mechanical complexity of large commercial RTS games, such as StarCraft II. At the same time, it retains the core difficulties, for example resource management, unit production, combat mechanics, and tactical positioning. MicroRTS is therefore suitable for the systematic investigation of learning methods. MicroRTS also contains numerous training challenges, such as large combinatorial action spaces, delayed rewards, multi-agent interactions, and it also requires robust strategies against different opponents. For example, the agent must decide early on whether it wants to play aggressively or defensively against the current opponent, which affects the overall game strategy and requires long-term planning.

Self-Play-Based Approaches. A central problem in RL is generalization across different opponents and strategies: agents often learn strategies that are strongly tailored to the training opponent or a specific game situation. This leads to overfitting and limited robustness. SP, Fictitious Self-Play (FSP), and LT are approaches that address this problem. In SP, the agent trains by playing against itself, which leads to a continuous learning process that does not require additional diverse or strong opponents. However, training is often unstable because of cyclical learning.

league training (LT). To reduce the instabilities of pure SP, the agent in FSP is not trained against the current agent but against previous versions of itself, which prevents cyclical learning [Heinrich et al., 2015]. LT extends FSP by training the agent against historical opponents from a league that consists of different agent types, for example policy snapshots from earlier training stages as well as specialized agents [Vinyals et al., 2019]. The goal is to stabilize the learning environment, in which agents compete against a broad spectrum of strategies and thereby become more robust.

Thesis Outline: Despite the theoretical advantages, the practical implementation of LT in complex environments such as MicroRTS is associated with significant challenges, that need to be addressed. These include:

- the efficient management and selection of opponents for training the Main Agent and for the specialized agents,
- stabilizing the training process under non-constant opponent distributions,
- dealing with large action spaces and sparse or delayed rewards, and

In addition, we will compare and evaluate the Results emperically. In particular, we will analyze the learning curve, strategic diversity, and robustness for the different training approaches.

Training the Base Agent: The base agent used in this project is trained in a three-stage procedure that combines BC and BC fine-tuning (BC-FT), followed by fine-tuning with PPO: The two BC phases enable the agent to quickly learn a competent base strategy by imitating expert behavior [Torabi et al., 2018]. This training approach reduces the initial, often inefficient exploration in DRL [Torabi et al., 2018]. Learning from scratch with DRL methods such as PPO can be very time-consuming [Martin and Jhala, 2021]. PPO is a well-known on-policy policy optimization method that clips the policy update size to stabilize policy updates [Schulman et al., 2017]. PPO further improves the performance of the BC agent through exploration and adaptation to the environment, allowing the agent to adapt and generalize beyond the expert strategies [Martin and Jhala, 2021]. This two-stage approach, in which a policy pre-trained with BC is refined by a DRL algorithm, has proven effective in reducing training time and improving overall performance compared to a pure DRL approach [Martin and Jhala, 2021].

2. Related Work

Game Playing Agents in MicroRTS The (unpublished) bachelor's thesis by M. Huft that underlies this work develops a competitive agent for MicroRTS and evaluates the hybrid training pipeline consisting of BC, BC-FT, and PPO for the base agent from the introduction [Huft, 2025].

Results and Limitations. The results show a clear performance increase across all three training phases (Table 1): BC mainly learns basic game mechanics, while BC-FT improves generalization against multiple bots. Finally, PPO achieves high win rates against many of the built-in bots [Huft, 2025]. There still are agents in the results against which the base agent does not perform well. The thesis also identifies limitations, in particular the focus on a single map and the limited opponent diversity, and proposes extensions using self-play or league mechanisms. Exactly this proposal is taken up by the present work by extending the existing base agent and the initial code with SP and LT. The opponents against which the evaluation is conducted include the MicroRTS built-in bots (RandomBiasedAI, WorkerRushAI, LightRushAI, and CoacAI).

Table 1. Winrates (%) for BC, BC-FT, and PPO reported in [Huft, 2025].

Bot	BC	BC-FT	PPO
WorkerRushAI	3	50	99
PassiveAI	98	100	100
LightRushAI	22	93	99
CoacAI	0	8	96
Mayari	0	5	95
RandomAI	52	95	99
RandomBiasedAI	18	68	86
rojo	0	34	84
mixedBot	4	44	59
izanagi	0	62	61
tiamat	11	78	84
droplet	0	31	59
guidedRojoA3N	4	42	75
naiveMCTS	0	5	3

MicroRTS-Py (Gym-μRTS) Huang et al. provide an efficient Gym-compatible interface for our environment MicroRTS [Ontañón, 2013] with *Gym-μRTS* / *MicroRTS-Py* [Huang et al., 2021]. MicroRTS-Py is an established and widely used DRL baseline that enables the investigation of DRL approaches in full real-time strategy game scenarios without requiring extensive technical infrastructure such as high-performance computing clusters [Huang et al., 2021].

Proximal Policy Optimization (PPO) training. The *openai/baselines* [Dhariwal et al., 2017] implementation of PPO was adapted in order to fit MicroRTS-Py. We use a large part of this implementation in LT [Huang et al., 2021]. It was also previously used in the PPO training phases of the base agent [Huft, 2025]. On top of openai/baselines, *invalid action masking* and suitable action/observation representations (e.g., GridNet [Han et al., 2019]) as well as architectural choices (such as IMPALA-CNN-a deep convolutional neural network structure with residual blocks for processing grid-based state representations in RL) [Espeholt et al., 2018] were added [Huang et al., 2021]. In a modified form, these are still used in the LT code.

GridNet. Huang et al. considered using *GridNet* or *Unit Action Simulation* (UAS) instead of GridNet. UAS iterates over all units at each step and returns the possible actions for each unit, which substantially reduces the action space. GridNet, in contrast, predicts an action for each cell on the map. This means that GridNet executes its large action space in a single step, but this also simplefies the training [Huang et al., 2021, Han et al., 2019]. Against the built-in bots of MicroRTS, which are based on heuristic rules, the PPO implementation with invalid action masking and IMPALA-CNN achieves an average win rate of 91% with UAS and 84% with GridNet. Ultimately, the decision was made to focus on GridNet, since its complexity is lower, and it facilitates compatibility with SP (and LT) [Huang et al., 2021]. SP was also evaluated in the work. However, there was a bug in the code that likely strongly affected the results for SP [Goodfriend, 2024].

RAISocketAI In 2023 RAISocketAI was the first DRL agent which won the IEEE MicroRTS Competition. RAISocketAI includes a re-implementation of MicroRTS-Py, which is used in our code. Among other things, data generation via replays is an important basis for BC [Huft, 2025], and a bug in the old implementation was fixed that marked non-owned resources as owned resources when the agent is player 2. According to Goodfriend, this likely caused SP in the previous MicroRTS-Py implementation to yield no improvements, The RAISocketAI implementation of FSP achieved an average win rate of 91% against the same MicroRTS built-in bots as in MicroRTS-Py, which is significantly higher than 84% in the previous implementation. However, FSP achieved only a 20% win rate against CoacAI, whereas the best previous implementations were almost always able to defeat CoacAI. With fine-tuning on CoacAI and Mayari, the average win rate against the built-in bots was increased to 98% [Goodfriend, 2024]. LT was not implemented or evaluated in the work.

BeClone *BeClone* demonstrated the effectiveness of BC with DRL fine-tuning in MicroRTS-Py using a two-phase training strategy for RTS agents: first, an initial policy is learned from experts via BC, and subsequently an RL fine-tuning is performed using *Advantage Actor-Critic* (A2C) [Martin and Jhala, 2021].

AlphaStar AlphaStar is a DRL agent for the complex RTS game StarCraft II, which was developed by DeepMind [Vinyals et al., 2019]. It demonstrates that LT can decisively contribute to robustness in complex RTS domains. The agent in AlphaStar was pre-trained with *imitation learning* from human replays.

Pre-League Training. In this thesis, we replace imitation learning with the BC phases and the PPO phase. Similar to AlphaStar our agent was further trained in a league of different agent types (Main Agent, Main Exploiters, League Exploiters, and historical snapshots (*historical agents*)) to reduce overfitting and forgetting and to promote diversity.

League Composition and Training. As in AlphaStar, in this thesis Main Exploiters are trained by training the base agent against all historical versions of the Main Agenten in the league, in order to identify and exploit weaknesses. At the same time, League Exploiters train against all historical agents. The Main Agent trains with *Prioritized fictitious self-play* PFSP against historical versions of itself, Main Exploiters, and League Exploiters. The prioritization of the historical opponents is computed based on the Main Agent's win rate against the respective historical agents. In this thesis, the prioritization formula was adjusted to better fit the MicroRTS environment. An exploiter is added to the league as a historical agent only when it shows a high win rate against all opponents or when a predefined step limit is reached [Vinyals et al., 2019].

Differences from AlphaStar. Although MicroRTS and StarCraft II substantially differ in complexity and representation, AlphaStar is an established reference design for stabilizing and generalizing SP through league-based matchmaking mechanisms and the inclusion of specialized exploiters [Vinyals et al., 2019]. AlphaStar performs policy updates using an *actor–critic method* with an off-policy correction mechanism called *V-trace* [Espeholt et al., 2018, Vinyals et al., 2019]. This approach is better suited for LT than PPO due to its robustness to off-policy data and its typically better scalability in large distributed setups. However, the implementation of PPO in MicroRTS-Py is significantly simpler and faster to realize, which is why PPO is a practical choice for implementing LT [Vinyals et al., 2019]. In addition, AlphaStar uses TD(λ) [Sutton and Barto, 1998] for value estimation value-function estimation.

3. Methods

3.1. MicroRTS-environment

MicroRTS is a two-player zero-sum game that is played on a two-dimensional grid (the map). Different units can be located on each cell of this map, and players can execute actions to move/train/attack units and to collect resources. The different units are:

Unit Types.

- **Base:**  The Base can store resources and train Workers. It is very important, since it is the only way to store resources. (Cost (in resources): 10, hit points (hp): 10)

- **Resources:**  There is only one type of resource. It can be stored by Workers in the Base. (Number of resources in a cell: depends on the map in our case 25)

- **Worker:**  Workers can collect resources and build Barracks. (Cost: 1, hp: 1, attack strength (at): 1)

- **Barracks:**  Barracks can train Light, Heavy, and Ranged units. (Cost: 5, hp: 4)

- **Light:**  Light units can move quickly, are trained quickly, and cost little. (Cost: 2, hp: 4, at: 2)

- **Heavy:**  Heavy units can move slowly are trained slowly and cost a lot, but have high at and hp. (Cost: 3, hp: 8, at: 4)

- **Ranged:**  Ranged units have an attack range of 3 cells. (Cost: 2, hp: 1, at: 1)

Worker, Light, Heavy, and Ranged units can move one cell in one of four directions. Units of player 1 are displayed with a blue outline, whereas units of player 2 are outlined in red. Movement is visualized with a gray line, attacking with a red line, and gathering with a green line.

Game Setup. Most games are decided within 2000 steps. Therefore, the maximum number of steps is set to 2000. The experiments in this thesis were conducted using the RAISocketAI re-implementation of MicroRTS and the map `basesWorkers16x16A`, which is shown in Figure 1. This symmetric map starts each player with one Base, five resources, two Workers, and 50 resources near the Base. **[Goodfriend, 2024]**

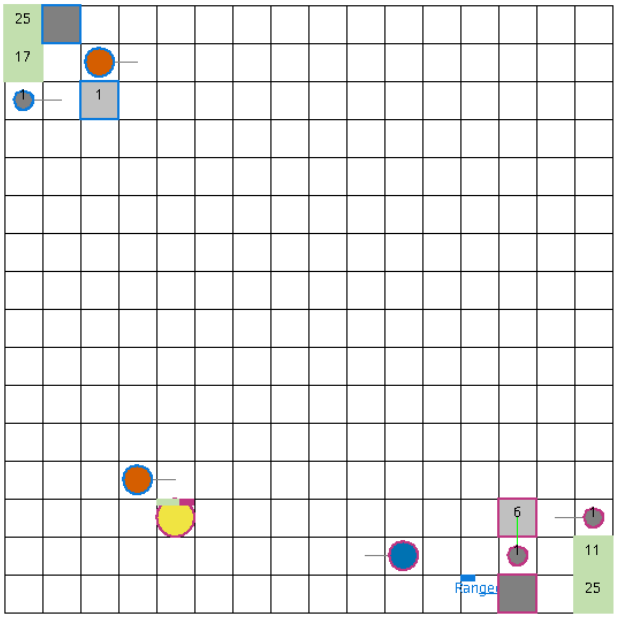

Figure 1. Screenshot `basesWorkers16x16A` map [Huft, 2025].

Action Space. The action space used in this thesis is implemented using the combination of GridNet and invalid action masking [Huang et al., 2021]. GridNet produces a unit action (an action for one specific cell) for each cell of the map, regardless of whether a unit is located on the cell and whether the action is valid [Han et al., 2019]. Building on this, invalid action masking is used to mask invalid actions, thereby also masking actions empty cells [Huang et al., 2021]. The *unit action space* is defined as an 8-dimensional vector per cell:

$$(\text{pos}, \text{type}, d_{\text{move}}, d_{\text{harvest}}, d_{\text{return}}, d_{\text{produce}}, \text{pType}, \text{relPos})$$

The components of the unit-action vector are defined as follows:

- **pos:** Linearized index of the cell in the grid (i.e., the grid represented in a one-dimensional space; e.g., for a 16×16 grid: range: [0, 255]).

- **type:** Action type (range: [0, 5], 0: Nop, 1: Move, 2 Harvest, 3: Return, 4: Produce, 5: Attack).

- $d_{\text{move}}, d_{\text{harvest}}, d_{\text{return}}, d_{\text{produce}}$**:** Directions (range: [0, 3], 0: North, 1: East, 2: South, 3: West) for the respective actions.

- **pType:** Type of the unit to be produced. (range: [0, 6], 0: Resource, 1: Base, 2: Barracks, 3: Worker, 4: Light, 5: Heavy, 6: Ranged).

- **relPos:** Relative attack position. (range: [0, 48]).

For our agent, the unit action space (without the position dimension) is linearized and represented using one-hot encoding. The position is implicitly encoded by the linear grid indexing of the GridNet output [Huang et al., 2021]. Given a map of size $h \times w$, the action space has shape $h \times w \times 78$. For our 16×16 map, the action space per environment in the GridNet representation would therefore contain $16 \times 16 \times 78 = 19968$ actions. However, many of them can be invalid and are masked by invalid action masking [Huang et al., 2021]. Afterward the position is incorporated before masking the actions, otherwise the masking would remove the structure of the Action-Space, which would remove the implicit encoding for the position. Depending on the action type, only parts of the action vector are relevant (e.g., if the action type is `Move`, then only the components `pos`, `type`, and $d_{\texttt{move}}$ are relevant (the other components are ignored)).

Observation Space. Observations are represented as a grid of shape $h \times w \times c$ per environment, where h and w are the height and width of the map, and c is the number of encoded features of each cell in the game state (e.g., unit types, hit points, move direction, etc.) [Huang et al., 2021]. The screenshot of the map basesWorkers16x16A 1 visualizes the observation space, without any additional information.

3.2. Training Pipeline

The training pipeline extended in this thesis consists of four phases: BC, BC-FT, PPO, and LT (or FSP/SP). The first three phases were already implemented in the previous work [Huft, 2025] and are only briefly described here. In BC/BC-FT, demonstrations from experts (here, the built-in bots CoacAI and Mayari) are used to train an initial policy to learn basic game mechanics/strategies [Huft, 2025].

- The BC phases reduces the initial, often inefficient exploration in DRL methods such as PPO, which can be very time-consuming when trained from scratch [Torabi et al., 2018, Martin and Jhala, 2021].
- The third phase improves the generalization and performance of the agent through additional exploration beyond expert demonstrations [Martin and Jhala, 2021]. PPO is an on-policy policy optimization method that constrains the size of policy updates in order to achieve stable training [Schulman et al., 2017].
- Using SP/FSP/LT, we aim to further generalize the base agent by training PPO not only against the built-in bots of MicroRTS (against which the base agent already achieves a high win rate), but also against opponents of equal strength or stronger. However, many opponents at this level of performance are not suitable for training, since they are very slow and thus slow down the collection of rollouts for PPO. In addition, the agent's generalization capability improves only to a limited extent when training against only a few new opponents, if the training environment does not provide sufficient opponent diversity. SP/FSP/LT address both problems by providing numerous opponents that continue to evolve over the course of training and achieve a high win rate against historical versions of the current agent.

3.3. self-play (SP)

In SP, the agent trains against its own current version, which leads to a continuous learning process with a co-evolving opponent [Silver et al., 2018].

Implementation. To implement SP in the MicroRTS-Py reimplementation Huang et al. implemented the environment to output the observations (obs) and rewards for both players, which allows the agent to train against itself by simply using the same policy for both players [Huang et al., 2021]. In RAISocketAI, that approach was improved and fixed [Goodfriend, 2024]. In this Thesis, the same approach with some adustments is used to implement SP, which is also used as a basis for FSP and LT.

Problems:

- The training is often unstable because of cyclical learning. For **example**, if policy B performs well against A, C performs well against B, and A performs well against C, the agent may be trapped in a cycle in which it repeatedly relearns only the policy that performs well against itself, but not against other opponents [Vinyals et al., 2019].
- A big problem in the SP training setup is that the agent has to learn to play as player 2 as well as player 1, which complicates the training and can lead to a slow learning process [Huang et al., 2021]. The biggest difference between player 1 and player 2 is that player 1 starts at the top left of the map, whereas player 2 starts at the bottom right. This means that the agent has to learn to start play in two different areas of the map, which can be difficult and time-consuming. The problem is further exacerbated by the fact the training of this thesis does not start from scratch, but from a base agent that was only trained as player 1, which means that the agent would have to unlearn the bias towards starting at the top left of the map and learn to start at the bottom right as well.

Solutions. FSP/PFSP are used to stabilize training and reduce cyclical learning. To address the second problem, the observations for player 2 are transformed by rotating them along by 180 degrees. From the agent's perspective, player 2 also starts at the top-left of the map. This makes the observations for player 1 and player 2 more similar and therefore makes SP with a pre-trained base agent possible, even though the base agent was exclusively trained as player 1.

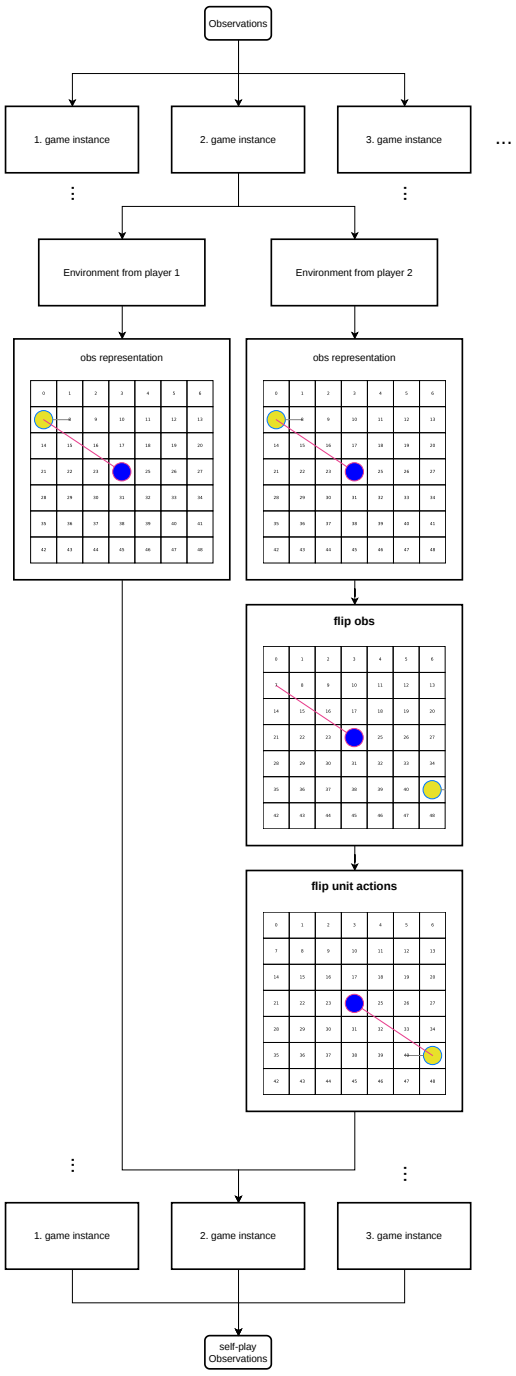

Figure 2. Observation adjustment for player 2 in SP. To rotate the observations, the GridNet representation was flipped for each player-2 environment. The unit actions in the observation were then flipped individually.

Actions are transformed accordingly, so that the environment still executes the intended actions for player 2. Masks must be transformed as well, since they are based on the environment's observations. The same rotation used for the observations is also applied to the invalid action masks. After masking, the selected actions (including a position component, that is added before masking, since masking removes the implicit positional structure of the GridNet action space) are transformed back to fit the MicroRTS-Py environment.

3.4. fictitious self-play (FSP)

To address the instabilities of pure SP, in FSP the agent is trained not exclusively against the current policy, but against a uniformly sampled pool of the agent's historical policies. This results in the learning agent learning a good response to the averaged opponent strategy, which reduces cyclical learning. Under standard assumptions, FSP can converge towards an approximate Nash equilibrium in two-player zero-sum games (with respect to the average strategies). Heinrich et al. also demonstrate this empirically in poker experiments [Heinrich et al., 2015].

Implementation. The implementation is similar to SP, but instead of Player 2 always using the current policy, Player 2 is sampled uniformly from a pool of historical agents (snapshots of the Main Agent at different training stages).

Problem with the implementation. If we implement in the described way, a problem arises: player 1 has priority in conflict resolution (e.g. when both players want to move to the same cell). This gives player 2 a disadvantage, although it has only a minor effect on the results. The actual problem is that the agent can exploit this prioritization in FSP if it learns that optimizing the training objective can be achieved primarily by improving player 1, while deliberately performing poorly as player 2. After sufficient training, the agent will learn a strategy that is based on player 1 being prioritized. Training then stagnates, since the agent already has a hight win rate against itself without develping a robust strategy against other opponents.

Example. The agent could learn to block an opponent in front of the Base by issuing a simultaneous move to the same target cell. In FSP, player 2 would then be blocked, while player 1 would not, allowing the agent to achieve a win rate close to 100% against itself.

Relevance for LT. In LT, the Result of the problem is not quite as problematic because there are exploiters that can be trained against, which remain adversarial to the Main Agent. However, a large portion of the training becomes ineffective and provides poor learning signals if the agent exploits the player 1 priority. On top of that the Problem is particularly relevant for LT, which combines SP and FSP, because only historical Agents are available for training in FSP, making the feedback for player 2 very sparse, while in LT the Main Agent can directly train against the current Main Agent. This yieldes more immediate feedback on the Main Agents performance as an opponent (player 2). As a result the agent no longer has to plan as far ahead in order to exploit this priority and minimize its own win rate as player 2.

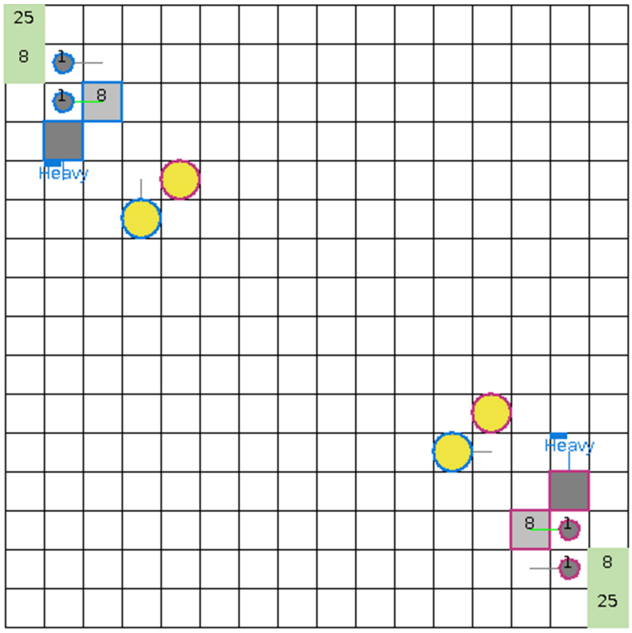

Figure 3. Screenshot showing an action conflict in the game. Both Heavy units of both players wanted to move to the same cell, as the opponent's Heavy units.

Solutions. In the re-implementation of MicroRTS-Py, there is a mode that resolves conflicts in a randomized manner. However, this does not help in this case, because even before the conflict arises, invalid action masking masks out the action of player 2 since it is invalid. To solve this problem, the learning agents in the parallel games are alternated between playing as player 1 and player 2. This does not prevent the prioritization of player 1. However, it can be shown empirically that the prioritization of player 1 has no major impact on the results. This prevents the exploitation of the prioritization by the learning agent, since it can now also be player 2 and therefore a strategy that is only effective for player 1 is no longer beneficial.

3.5. prioritized fictitious self-play (PFSP)

LT uses an extended variant of FSP called PFSP [Vinyals et al., 2019]. PFSP extends FSP by selecting opponents against which the agent can train efficiently, rather than selecting opponents uniformly. The selection probability is based on the win rate of the learning agent against the potential opponent [Vinyals et al., 2019].

Prioritization Function. The exact prioritization function can differ for different agent types. In AlphaStar, for the Main Agent, opponents of medium difficulty (win rate close to 50%) are prioritized. While exploiters prioritize the strongest opponent (against which the win rate is closest to 0%). Adapted prioritization functions tailored to our training setup are used in this thesis, which deviate from those used in AlphaStar. An important difference is that training against an opponent against which the base agent has a win rate close to 0% yields little informative gradient or learning signal.

Implemented Prioritization Function. To avoid opponents resulting in little learning signals, the following prioritization functions was used for the Exploiters:

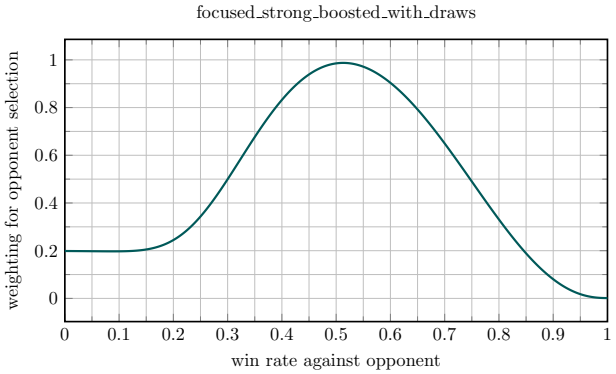

Figure 4. Prioritization function for the Exploiters in PFSP.

As can be seen in the figure, we prioritize opponents with a low win rate not close to 0%. Furthermore, opponents with a win rate of 0% are prioritized over those with a win rate of 100%, since there is substantial learning potential for an opponent with a win rate of 0%, whereas a win rate of 100% usually means that the agent already defeats the opponent reliably. In this thesis PFSP was modified so that all agents have a non-zero probability of being selected. This ensures that no opponent is completely ignored because it lost all initial games (yielding a 100% win rate for the learning agent), since these initial results may have been outliers.

3.6. league training (LT)

LT combines and extends SP and PFSP to further stabilize training. It also promotes robustness by training against opponents sampled from a *league*, including Main Exploiters and League Exploiters and Historical Agents [Vinyals et al., 2019].

League composition and roles. Learning Agents of the league were trained in parallel. Similar to AlphaStar [Vinyals et al., 2019], the league in this thesis consists of the following agent types:

- **Main Agent (MA):** The primary policy that is optimized and evaluated.
- **Historical Agents:** Frozen snapshots of earlier MA or exploiter agent policies. They stabilize training by providing a distribution of past strategies and mitigate catastrophic forgetting.
- **Main Exploiters (ME):** Agents trained only against Historical Main Agents, which encourages them to identify and exploit weaknesses in the Main Agent.
- **League Exploiters (LE):** Agents trained against all Historical Agents, which further promotes diversity by identifying systemic weaknesses of the league.

Historical Agents are the only agent type that is not a learning agent. The main differences between the roles are the opponent-sampling distribution and the hyperparameters. All agents share the same observation/action representation (GridNet with invalid action masking) and use the same underlying PPO algorithm, but are trained with role-specific hyperparameters (e.g., learning rate and PFSP prioritization function).

Training loop and snapshot management. LT iterates through cycles consisting of the following phases:

1. **Select opponent:** Sample an opponent from the league.
2. **Collect rollouts:** Run matches between the learning agent and the sampled opponent for a fixed number of steps. No agent is updated during this phase.
3. **Update policy:** Perform PPO updates using the collected trajectories against the sampled opponent.
4. **Update statistics:** Periodically evaluate against a subset of league members to update win rate estimates used by PFSP.

5. **Add snapshots:** At predefined intervals (or when performance criteria are met), store a frozen copy of the current agent as a new Historical Agent.

A phase of the training loop is considered completed only after every agent type (MA, ME, LE) has completed the phase. For example, the Main Agent only proceeds to the next phase after all Main Exploiters and League Exploiters have completed the current phase.

Bot environments. Bot environments are added to the league to stabilize training by continuing the PPO training against the built-in MicroRTS bots CoacAI and Mayari, which the base agent was trained against. Only the Main Agent trains against the built-in bots, since learning signals from built-in bots could interfere with identifying and exploiting weaknesses in the Main Agent or the league, which is the main purpose of the exploiters (e.g., if the built-in bots are too strong against an exploit that the Main Agent has not yet trained against). In MicroRTS-Py the environments for built-in bots and for SP are not interchangeable, since the built-in bots are not implemented as neural policies and therefore cannot be used in SP/FSP/LT directly. The number of parallel environment instances and their built-in bot opponent (if any) have to be defined at initialization of the MicroRTS-Py environments, which means the number of environments for each built-in bot remains fixed. One of the main advantages of LT is that training against opponents with bad learning signals is reduced. To retain that improvement we want the number of bot environments to be dynamic. Therefore, the number of bot environments is adjusted dynamically by reinitializing a MicroRTS instance, which is only used for training against the built-in bots and not for SP/FSP/LT (because reinitialization also means resetting the environments in the MicroRTS instance). The MicroRTS instance for built-in bots executes its step function at the same time as the SP/FSP/LT instance.

number of parallel bot environment instances. Two hyperparameters define the maximum number of parallel bot environment instances and the minimum number of parallel bot environment instances. Among other things, references to the observation, action, and invalid action mask buffers must be adjusted to the correct size and partially reset whenever the number of parallel bot environment instances is changed.

Summary: Opponent Selection. PFSP samples the next opponent from a given set of possible opponents, with probabilities based on the current learning agent's win rates against the potential opponents, whereas SP always trains against the current learning Agent. The opponent selection for the League is illustrated in Figure 5. Depending on the agent role, opponents are either the current Main Agent or sampled from specific subsets of historical agents using PFSP. The Main Agent also trains against additional bot environments, while Main Exploiters and League Exploiters are matched only against agents from the league.

Resetting Exploiters. The AlphaStar paper [Vinyals et al., 2019] describes that exploiters reset their weights to the initial values (here: the Base Agent). In our approach exploiter resets also reset the optimizers, game statistics (e.g., win rates so that PFSP does not use outdated information) and the unit bonus distribution (if applicable). Furthermore exploiters only reset after a game has ended. As a result, an exploiter can reset during a rollout. If that happens, then the rollout contains trajectories from the old policy and therefore cannot be used for the on-policy training in our PPO implementation. To keep the training as simple as possible, the entire PPO update for that rollout after an exploiter reset is skipped.

AlphaStar Pseudocode. In addition to the modifications made to adapt the pseudocode to the MicroRTS environment the following corrections to the AlphaStar pseudocode in the original paper [Vinyals et al., 2019] were made:

- When resetting Exploiters, the pseudocode first resets the weights and only then saved the snapshot for the Historical Agent. In our implementation, this order is reversed so that the snapshot is not taken from the reset Exploiter.
- Main Exploiters were only trained against the current Main Agent, if they had a high win rate against the Main Agent from the very beginning. As soon as this win rate dropped, the Main Exploiter stopped training against the Main Agent for the remainder of this reset cycle. However, in the pseudocode, Main Exploiters were only reset/checkpointed (thereby creating a Historical Main Exploiter) if the win rate against the Main Agent was high. This means that a Main Exploiter would never reset if it initially had a low win rate against the Main Agent. Since the current Main Exploiter has already trained against the Base Agent, it is very likely that it will never checkpoint before the maximum number of steps is reached, even if it has a high win rate against all historical Main Agents. To solve this problem, the win rate against the Main Agent is replaced by the minimum win rate against all historical Main Agents. This ensures that the Main Exploiter is reset and checkpointed as soon as it achieves sufficiently high win rates against the available historical Main Agents (making high win rate

against the Main Agent likely) and no longer has much to learn from the available opponents.

- Another change to the pseudocode is that win rates are initialized to 0.5 for the initial games. This makes it less likely that the agent will ignore opponents it has not yet played against because of outlier results with very high or very low win rates, including the Main-Exploiter, Main-Agent matchup.

3.7. Additional modifications for diversity

The biggest problem with the training setup is the lack of opponent diversity. To make exploiters more diverse, exploration can be increased with hyperparameters such as the learning rate, the entropy coefficient, and the KL regularization coefficient.

Problems with overall increased exploration. Excessively increasing exploration for exploiters can lead to worse performance and longer training times. We hypothesize that is because the starting point (the Base Agent) is already very strong and effective strategies differ substantially between unit types, so that the agent must unlearn a very large portion of the current strategy before it can learn new strategies for other unit types. In between the unlearning and learning phases, the agent performs very poorly, which leads us to believe that there are very distant clusters (corresponding to different unit types) of local maxima in the reward landscape. This could explain why finding different maxima with the same unit type is possible, but changing to a strategy for different unit types is very difficult and time-consuming. Furthermore, the same problem occurs for every exploiter, which compounds the problem. What makes diversity even more difficult is that, in a complex environment such as MicroRTS, league training will most likely not cover all possible Main Agent weaknesses that can be exploited by an exploiter using Heavy Units (the unit type strongly favored by the Base Agent). This suggests that, even after training, there will remain weaknesses that are easier to exploit than those that require shifting the focus to a different unit type. For example, if the Base Agent, which favors Heavy Units, is to exploit a weakness that occurs when the Main Agent is attacked by many Worker units, then throughout the episode, every Worker on the field must refrain from building Barracks, since this would waste too many resources for the strategy to work. Instead of using only two Workers, the Agent has to produce as many as possible, while every unit but two rushes the opponent. The Agent only receives a positive reward if all of this happens in the same game, making this behavior unlikely to occur. The second strategy in the example is simmilar to a built-in bot strategy of MicroRTS called WorkerRushAI.

Production Exploration Bonus. To further increase exploration without increasing training time too much, an exploration bonus targeting the production of Workers, Ranged, Heavy, and Light Units can be added to the reward. By targeting the production of different unit types, the agent is encouraged to explore different strategies for different unit types, which can lead to more diverse exploiters. This was achieved by using a mask that gives Barracks and Bases (units whose only function is to produce other units) an entropy bonus, which encourages more diverse production decisions and thereby increases exploration across different unit types, by making the agent interact with them. While this helped to increase exploration of a broader range of unit types, as reflected by the fact that the agent uses a wider variety of units, it was not enough to make the agent explore strategies for different unit types.

Unit Bonus. To address the problem of insufficient diversity, a clipped unit bonus can be added to the reward, encouraging the agent to use different unit types (e.g., by giving the Base Agent a bonus every time it produces a Ranged Unit). The number of other units types increased when using the Unit Bonus. However, even with weaker agents such as the agent after the BC phase and over 20 million steps, the agent did not develop strategies centered on different unit types and instead produced the unit type with a bonus at the end of a round, when the result was already decided.

Unit Bonus Distribution. Unit Bonus Distribution is an improved method for increasing the diversity of exploiters based on the Unit Bonus. The idea is to continue training the agent with a new feature that receives the Unit Bonus sampled from a distribution that is updated every game. The trained agent can then be used as an Exploiter with a static bonus or a distribution-based bonus, resulting in an Exploiter with definable tendencies toward different unit types. This also allows the retraining of the BC and PPO phases with the Unit Bonus, because only one agent needs to be trained instead of multiple agents with different fixed Unit Bonuses, which would be very time-consuming. Retraining the agent from scratch with the Unit Bonus Distribution showed promising results. However, the training was not continued long enough to evaluate the effect of the Unit Bonus Distribution on the diversity of the exploiters, since training with the Unit Bonus Distribution is very time-consuming because the BC phase did not work with the Unit Bonus Distribution method and therefore had to be skipped. Continuing the training with the Base Agent had the same problem as training with the Unit Bonus.

Annealed Entropy Coefficient. The Annealed Entropy Coefficient decreases the weight of the entropy bonus in the

loss function and thereby decreases the exploration of the agent over the course of training. This was implemented for the exploiters to further increase exploration during the early training steps, while retaining a more deterministic policy in the later stages of training. This coefficient resets when the corresponding Exploiter resets. In the case of a League Exploiter, it is possible that the exploiter checkpoints but does not reset. If that happens, the Annealed Entropy Coefficient is reinitialized to half of its previous initial value.

New Initial Agents. To address the problem of insufficient diversity we created new initial agents that are focused on different unit types, which can be used as the initial Main Exploiters in the League Training. This makes it possible for the Agent th exploit weaknesses related to different unit types, without having to unlearn the strategy for the unit type favored by the Base Agent first, which can lead to more diverse exploiters in less training steps. We only created new initial agents for the Main Exploiters, because we want to focus on training the already strong Base Agent further. The new initial Agents were not trained long enough, to compare against the Base Agent, but they are strong enough to identify weaknesses in the Main Agent and thereby improve generalization.

3.8. Delta Score.

To better lead the agent through the sparse reward landscape of MicroRTS, we added the change in score from MicroRTS [Huang et al., 2021,Ontañón, 2013] between two time steps to the reward. The score for each environment is calculated based on the current game state, which includes the number and type of units, their hit points, and the resources of both players. The Delta Score is the difference between the score at the current time step and the score at the previous time step, which makes it a good immidiet learning signal for the agent. The Score does not always accuratly predict how well an agent is playing, that is why an coefficient is used to scale the Delta Score, so that the weighted score over an game is a lot smaller than the win/loss reward. In the prior work from M. Huft [Huft, 2025] the agent we now use as the Base Agent was trained with an attack bonus, which is a reward for attacking the opponent's units. The attack bonus was removed for our training and we adjusted the Delta Score in the initial games to be approxemetly as big as the attack bonus would have been.

4. Experimente

5. Conclusion

References

[Dhariwal et al., 2017] Dhariwal, P., Hesse, C., Klimov, O., Nichol, A., Plappert, M., Radford, A., Schulman, J., Sidor, S., Wu, Y., and Zhokhov, P. (2017). Openai baselines. https://github.com/openai/baselines. 2

[Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). Impala: scalable distributed deep-rl with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1406–1415. 2, 3

[Goodfriend, 2024] Goodfriend, S. (2024). Raisocketai: The first deep reinforcement learning agent to win the ieee microrts competition. In *2024 IEEE Conference on Games (CoG)*, pages 1–8. 3, 4, 5

[Han et al., 2019] Han, L., Sun, P., Du, Y., Xiong, J., Wang, Q., Sun, X., Liu, H., and Zhang, T. (2019). Grid-wise control for multi-agent reinforcement learning in video game AI. In Chaudhuri, K. and Salakhutdinov, R., editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2576–2585. PMLR. 2, 3, 4

[Heinrich et al., 2015] Heinrich, J., Lanctot, M., and Silver, D. (2015). Fictitious self-play in extensive-form games. In Bach, F. and Blei, D., editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 805–813. 1, 7

[Huang et al., 2021] Huang, S., Ontanón, S., Bamford, C., and Grela, L. (2021). Gym-microrts: Toward affordable full game real-time strategy games research with deep reinforcement learning. *CoRR*, abs/2105.13807. 1, 2, 3, 4, 5, 11

[Huft, 2025] Huft, M. (2025). Game playing agents in microrts. 2, 3, 4, 5, 11

[Martin and Jhala, 2021] Martin, D. and Jhala, A. (2021). Beclone: Behavior cloning with inference for real-time strategy games. In *Joint Proceedings of the AIIDE 2021 Workshops*. 2, 3, 5

[Ontañón, 2013] Ontañón, S. (2013). The combinatorial multi-armed bandit problem and its application to real-time strategy games. In *Proceedings of the Ninth Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 9, pages 58–64. AAAI. 2, 11

[Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR*, abs/1707.06347. 2, 5

[Silver et al., 2018] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362:1140–1144. 5

Table 2. Hyperparameters used for the League Training setups

Category	Parameter	Value	Description
League Setup			
League	num_main_exploiters	4	Number of Main Exploiters.
League	num_league_exploiters	2	Number of League Exploiters.
League	selfplay_ready_save_interval	$5 \cdot 10^5$	Step interval for potential league checkpoints. (checkpoint if the win rate is sufficiently high)
League	main_selfplay_save_interval	$6 \cdot 10^6$	Step interval for forced Main-Agent checkpoints.
League	main_exploiter_selfplay_save_interval	$9 \cdot 10^6$	Step interval for forced Main-Exploiter checkpoints.
League	league_exploiter_selfplay_save_interval	$9 \cdot 10^6$	Step interval for forced League-Exploiter checkpoints.
League	main_winrate_threshold	0,7	win rate threshold for Main Agent checkpoints.
League	main_exploiter_winrate_threshold	0,7	win rate threshold for Main Exploiter checkpoints.
League	league_exploiter_winrate_threshold	0,7	win rate threshold for League Exploiter checkpoints.
League	main_exploiter_vs_main_winrate_threshold	0,3	Threshold for Main Exploiters against the Main Agent.
PFSP	pfsp_min_prob_factor	0,25	Minimum probability factor added to PFSP.
PFSP	main_PFSP_prob	0,5	Probability of using PFSP with historicals for the Main Agent.
PFSP	main_SP_prob	0,35	Probability of SP for the Main Agent.
Environments and Training Scope			
Environments	num_bot_envs	5	Initial number of bot environments.
Environments	min_num_bot_envs	5	Lower bound for the number of bot environments.
Environments	max_num_bot_envs	10	Upper bound for the number of bot environments.
Environments	min_bot_winrate	0,955	Lower win rate threshold for bot-environment adjustment. (if crossed num_bot_envs == max_num_bot_envs)
Environments	max_bot_winrate	0,98	Upper win rate threshold for bot-environment adjustment. (if crossed num_bot_envs == min_num_bot_envs)
Environments	num_main_envs	25	Number of parallel environments for the Main Agent.
Environments	num_envs_per_main_exploiters	25	Number of environments per Main Exploiter.
Environments	num_envs_per_league_exploiters	25	Number of environments per League Exploiter.
Training	total_timesteps	200 000 000	Total number of training steps.
Training	checkpoint_end_buffer_steps	$3 \cdot 10^6$	Exploiters cannot checkpoint checkpoint_end_buffer_steps steps bevor reaching total_timesteps .
Reward Function			
Reward	attack_reward_weight	0,0	Weight of the attack reward.
Reward	rewardscore	0,002	Scaling factor of Delta Score.
PPO Hyperparameters (Main Agent)			
PPO Main Agent	PPO_learning_rate	$5 \cdot 10^{-5}$	Learning rate of the Main Agent.
PPO Main Agent	num_steps	512	Number of steps per rollout.
PPO Main Agent	gamma	1,0	Discount factor.
PPO Main Agent	gae_lambda	0,95	GAE parameter.
PPO Main Agent	ent_coef	0,01	Entropy regularization coefficient.
PPO Main Agent	vf_coef	0,15	Weight of the value loss.
PPO Main Agent	max_grad_norm	0,5	Gradient clipping threshold.
PPO Main Agent	clip_coef	0,15	PPO clipping coefficient.
PPO Main Agent	update_epochs	4	Number of update epochs per PPO cycle.
PPO Main Agent	kle_stop	true	Early stopping when KL divergence becomes too large.
PPO Main Agent	kle_rollback	false	No rollback when the KL threshold is exceeded.
PPO Main Agent	target_kl	0,01	Target KL-divergence value.
PPO Main Agent	kl_coeff	0,3	Coefficient of the KL penalty term.
PPO Main Agent	norm_adv	true	Advantages are normalized.
PPO Main Agent	anneal_lr	true	Learning-rate annealing is enabled.
PPO Main Agent	clip_vloss	true	Value-loss clipping is enabled.
PPO Hyperparameters (Exploiters) (only differences to Main Agent hyperparameters)			
PPO Exploiter	exploiter_ent_coef	0,015	Entropy regularization coefficient of the Exploiters.
PPO Exploiter	exploiter_vf_coef	0,1	Weight of the value loss for the Exploiters.

Listed are the hyperparameters and control parameters used in this thesis. Some Parameters (e.g., model paths, project names, or logging names) were omitted for clarity.

[Sutton and Barto, 1998] Sutton, R. S. and Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. The MIT Press. 3

[Torabi et al., 2018] Torabi, F., Warnell, G., and Stone, P. (2018). Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954*. 2, 5

[Vinyals et al., 2019] Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., Leblond, R., Pohlen, T.,

Table 3. Training parameters for PPO training.

Parameter	**Value**
Initial learning rate (Adam)	2.5×10^{-5}
Rollout steps per environment	512
Minibatch Size	3072
Max episode length	2,000 timesteps
Discount factor γ	1
Value function coefficient β	0.01
Entropy coefficient η	0.005
Gradient norm threshold ω	0.5
KL regularization coefficient λ_{KL}	0.4
GAE parameter λ	0.95

Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T. L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T., Kavukcuoglu, K., Hassabis, D., Apps, C., and Silver, D. (2019). Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354. 1, 3, 5, 8, 9

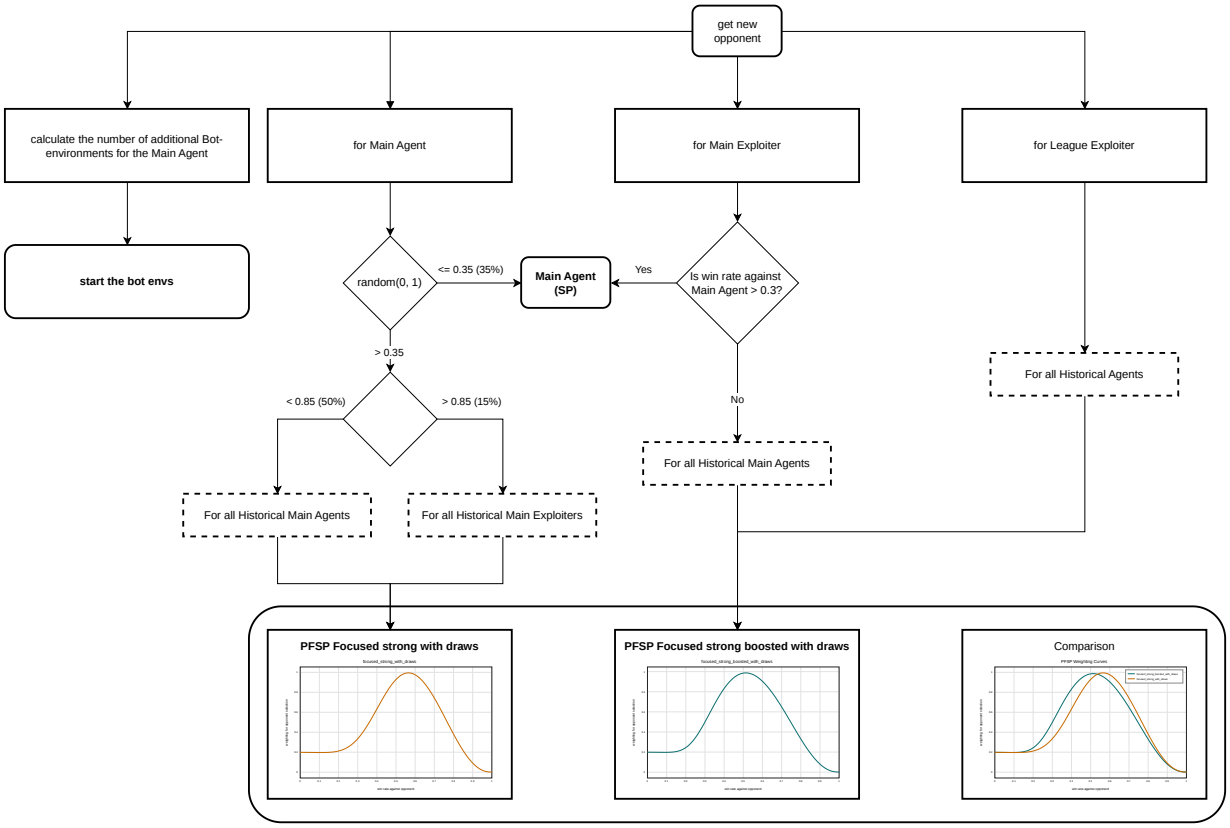

Figure 5. Matchmaking process for the league. Dashed boxes are candidate opponent sets, while the percentages on the arrows are fixed sampling probabilities. The PFSP curve gives the weighting for the opponent win rate

Erklärung

GPT!! Hiermit versichere ich, dass ich die Bachelorarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Düsseldorf, den XXXXXXXXXXX

Name!!!